

I

V1

# ECG disease classification

## Чекпоинт 5

### Нелинейные ML-модели

aVR

V4

aVL

V5

aVF

V6

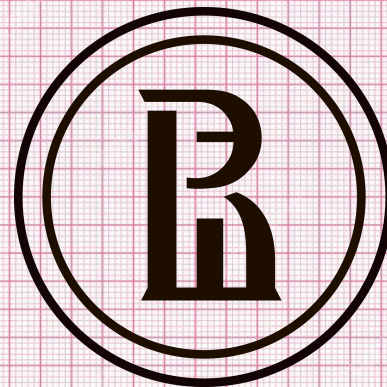
Куратор:

Карагодин Никита

Команда № 95

Пигальский Максим Валерьевич

Родионов Никита Андреевич



# Описание проекта

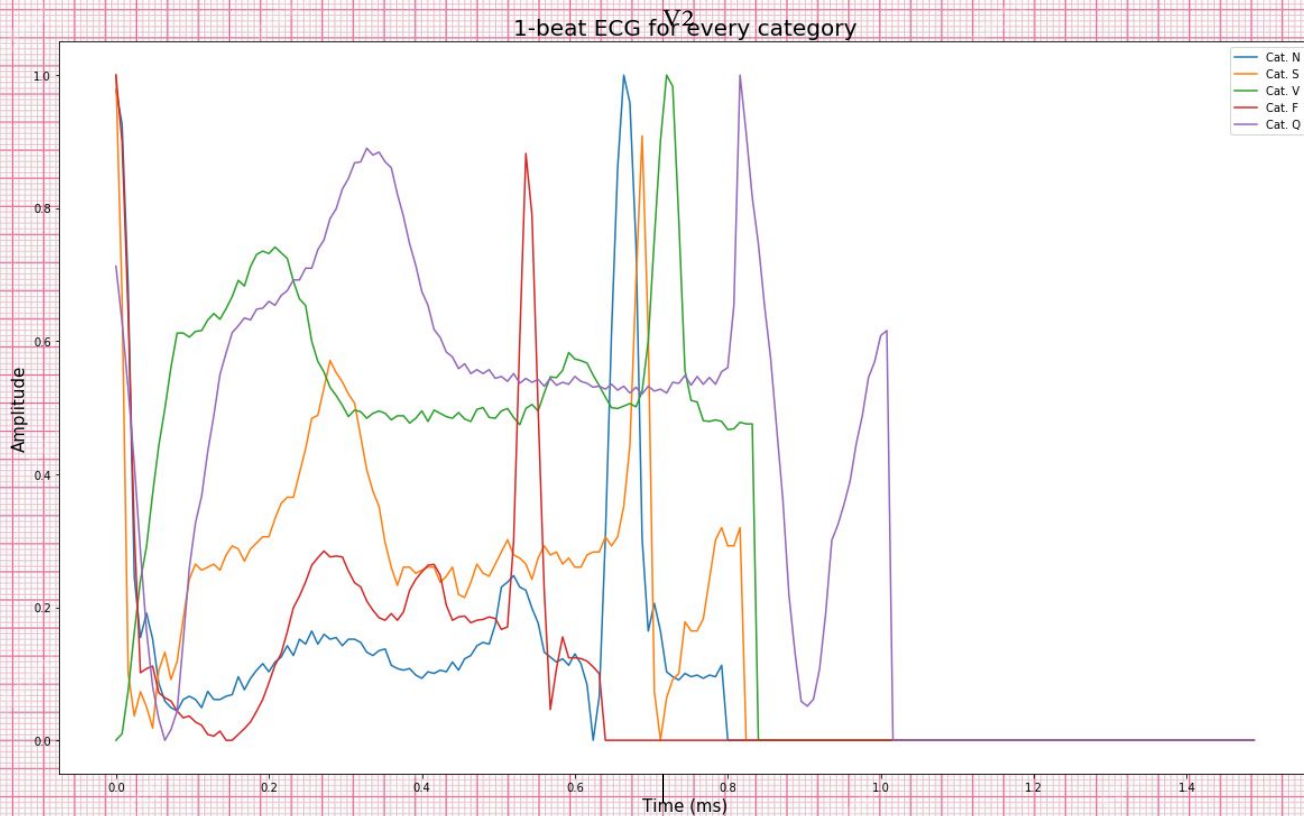
В данном проекте предлагается использовать открытые данные электрокардиограмм (ЭКГ) для анализа здоровья человека. Эти данные используются для разработки моделей машинного обучения, которые выявляют и классифицируют заболевания сердца, используя данные ЭКГ.

В данной презентации представлены результаты применения классических нелинейных ML-алгоритмов.



25 mm/sec 10 mm/mV

# Сэмплы каждого класса



25 mm/sec 10 mm/mV

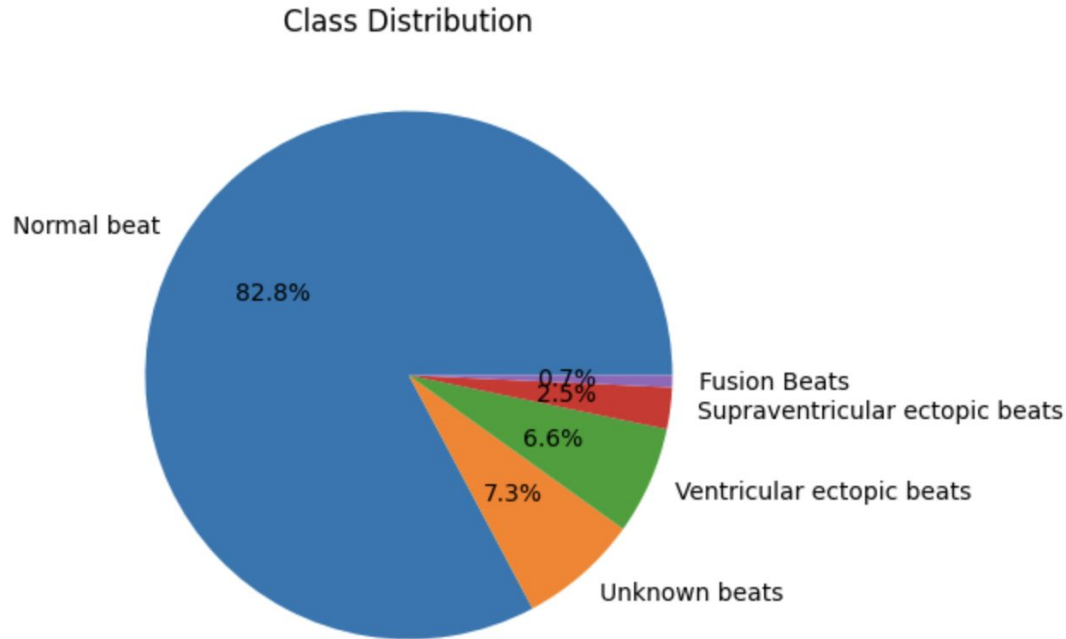
# Преобразование данных

ЭКГ содержит сложные, динамичные и высокочастотные паттерны, которые могут быть искажены или потеряны при ручном извлечении признаков, что является недопустимым в медицинском домене. Такие тонкие различия сложно выразить через привычные статистики (например, среднее, дисперсия, частота пиков).

Индустриальные/SOTA решения анализа ЭКГ опираются на глубокие модели, анализируя “сырые” ряды: морфология волн (P, QRS, T) и их взаимосвязи отлично улавливаются CNN или RNN. В случае применения ансамблей деревьев оптимальным будет положиться на автоматические преобразования данных внутри модели по описанным выше причинам.



# Распределение классов



В связи с дисбалансом классов в сторону “нормального ритма” нашей основной метрикой качества станет F1-Score (macro).

# План работы

1. Протестируем стандартные реализации древесных алгоритмов библиотеки `scikit-learn` (`DecisionTreeClassifier`, `RandomForestClassifier`, `GradientBoostingClassifier`);
2. Протестируем реализации градиентного бустинга библиотек `CatBoost` и `XGBoost`, оптимизируем гиперпараметры с помощью `Optuna`;
3. Сравним полученные результаты, сделаем выводы о применимости алгоритмов классического ML.



# Decision Tree & Random Forest

Sklearn DecisionTreeClassifier (default parameters)

Tree model report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.97      | 0.97   | 0.97     | 18117   |
| 1            | 0.60      | 0.61   | 0.60     | 556     |
| 2            | 0.87      | 0.86   | 0.87     | 1448    |
| 3            | 0.54      | 0.60   | 0.57     | 162     |
| 4            | 0.93      | 0.94   | 0.93     | 1608    |
| accuracy     |           |        | 0.95     | 21891   |
| macro avg    | 0.78      | 0.79   | 0.79     | 21891   |
| weighted avg | 0.95      | 0.95   | 0.95     | 21891   |

Sklearn RandomForestClassifier (default parameters)

Forest model report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.97      | 1.00   | 0.99     | 18117   |
| 1            | 0.98      | 0.59   | 0.73     | 556     |
| 2            | 0.98      | 0.88   | 0.93     | 1448    |
| 3            | 0.91      | 0.61   | 0.73     | 162     |
| 4            | 0.99      | 0.94   | 0.97     | 1608    |
| accuracy     |           |        | 0.97     | 21891   |
| macro avg    | 0.97      | 0.80   | 0.87     | 21891   |
| weighted avg | 0.97      | 0.97   | 0.97     | 21891   |

# Gradient Boosting & comparison

## Sklearn GradientBoostingClassifier (default parameters)

Gradient Boosting model report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.97      | 1.00   | 0.98     | 18117   |
| 1            | 0.93      | 0.56   | 0.70     | 556     |
| 2            | 0.95      | 0.82   | 0.88     | 1448    |
| 3            | 0.79      | 0.52   | 0.63     | 162     |
| 4            | 0.98      | 0.93   | 0.95     | 1608    |
| accuracy     |           |        | 0.96     | 21891   |
| macro avg    | 0.93      | 0.76   | 0.83     | 21891   |
| weighted avg | 0.96      | 0.96   | 0.96     | 21891   |

| Model             | F1-macro |
|-------------------|----------|
| Decision Tree     | 0.79     |
| Random Forest     | 0.87     |
| Gradient Boosting | 0.83     |



# Оптимизация гиперпараметров

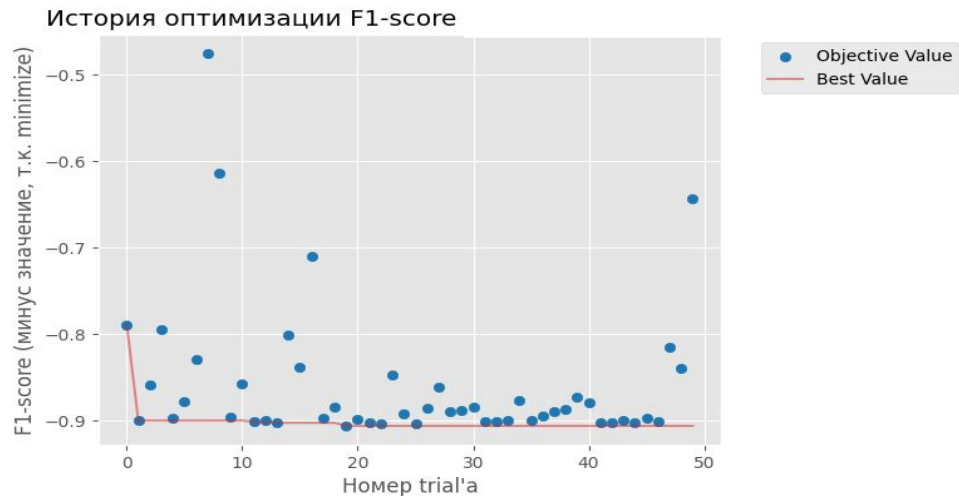
## CatBoost + Optuna

### Best CatBoost params:

```
{  
  'iterations': 869,  
  'depth': 9,  
  'learning_rate': 0.09346318063544509,  
  'random_strength': 0.009280683817626017,  
  'bagging_temperature': 0.17225843483572442,  
  'border_count': 134,  
  'l2_leaf_reg': 0.9449894803823506  
}
```

Best CatBoost F1-Score macro: 0.9060

|    | iterations | depth | learning_rate | random_strength | bagging_temperature | border_count | l2_leaf_reg | F1-Score |
|----|------------|-------|---------------|-----------------|---------------------|--------------|-------------|----------|
| 19 | 869        | 9     | 0.0935        | 0.0093          | 0.1723              | 134          | 0.9450      | 0.9060   |
| 25 | 871        | 7     | 0.1003        | 0.0032          | 0.0608              | 118          | 0.9038      | 0.9035   |
| 22 | 753        | 7     | 0.1706        | 0.0169          | 0.0560              | 147          | 0.8664      | 0.9031   |
| 13 | 990        | 7     | 0.2926        | 0.1269          | 0.0888              | 52           | 0.3730      | 0.9025   |
| 44 | 820        | 10    | 0.1007        | 0.0860          | 0.0832              | 83           | 0.4618      | 0.9024   |
| 41 | 925        | 8     | 0.1704        | 0.0345          | 0.0722              | 143          | 0.8103      | 0.9022   |
| 42 | 903        | 8     | 0.1774        | 0.0382          | 0.0339              | 146          | 0.3606      | 0.9022   |
| 21 | 910        | 8     | 0.1755        | 0.0294          | 0.0640              | 144          | 0.7199      | 0.9018   |
| 46 | 747        | 10    | 0.1220        | 0.1165          | 0.4325              | 47           | 0.4237      | 0.9015   |
| 11 | 963        | 8     | 0.2937        | 0.1074          | 0.1102              | 84           | 0.4787      | 0.9015   |



# Оптимизация гиперпараметров

## XGBoost + Optuna

### Best XGBoost params:

```
{  
'n_estimators': 343,  
'depth': 6,  
'learning_rate': 0.28586084392864963,  
'subsample': 0.933337325928994,  
'colsample_bytree': 0.6229576991062298,  
'reg_alpha': 0.008965101255486945,  
'reg_lambda': 1.5389069714056776,  
'gamma': 0.0021126264863443865  
}
```

Best XGBoost F1-Score macro: 0.9098

|    | n_estimators | max_depth | learning_rate | subsample | colsample_bytree | reg_alpha | reg_lambda | gamma  | F1-Score |
|----|--------------|-----------|---------------|-----------|------------------|-----------|------------|--------|----------|
| 18 | 343          | 6         | 0.2859        | 0.9333    | 0.6230           | 0.0090    | 1.5389     | 0.0021 | 0.9098   |
| 31 | 607          | 6         | 0.1995        | 0.9118    | 0.8858           | 0.0249    | 1.2403     | 0.0142 | 0.9095   |
| 21 | 335          | 5         | 0.2756        | 0.8672    | 0.5662           | 0.0012    | 2.2607     | 0.0013 | 0.9094   |
| 26 | 389          | 6         | 0.2104        | 0.9600    | 0.5543           | 0.1010    | 2.2429     | 0.0196 | 0.9082   |
| 41 | 430          | 6         | 0.1708        | 0.6938    | 0.9077           | 0.0091    | 4.9612     | 0.0426 | 0.9079   |
| 32 | 420          | 6         | 0.1794        | 0.7804    | 0.9253           | 0.0080    | 5.0904     | 0.0395 | 0.9073   |
| 42 | 459          | 6         | 0.2085        | 0.6610    | 0.8916           | 0.0099    | 5.9115     | 0.0152 | 0.9068   |
| 46 | 432          | 6         | 0.2374        | 0.6313    | 0.8106           | 0.0138    | 0.8322     | 0.0551 | 0.9067   |
| 14 | 603          | 6         | 0.2892        | 0.9151    | 0.8571           | 0.0402    | 1.0015     | 0.0122 | 0.9062   |
| 40 | 791          | 4         | 0.1328        | 0.8119    | 0.5240           | 0.0253    | 0.4532     | 0.0066 | 0.9062   |





# Выводы

1. Лучшими алгоритмами оказались бустинги с оптимизированными гиперпараметрами (в частности XGBoost);
2. Метрики XGBoost следует использовать в качестве финального бейзлайна по результатам применения классических алгоритмов ML;
3. Хотя нам удалось установить достаточный threshold для бейзлайн модели ( $> 0.9$ ), для эффективного решения задач нашего проекта необходимо использовать глубокие нейронные сети.